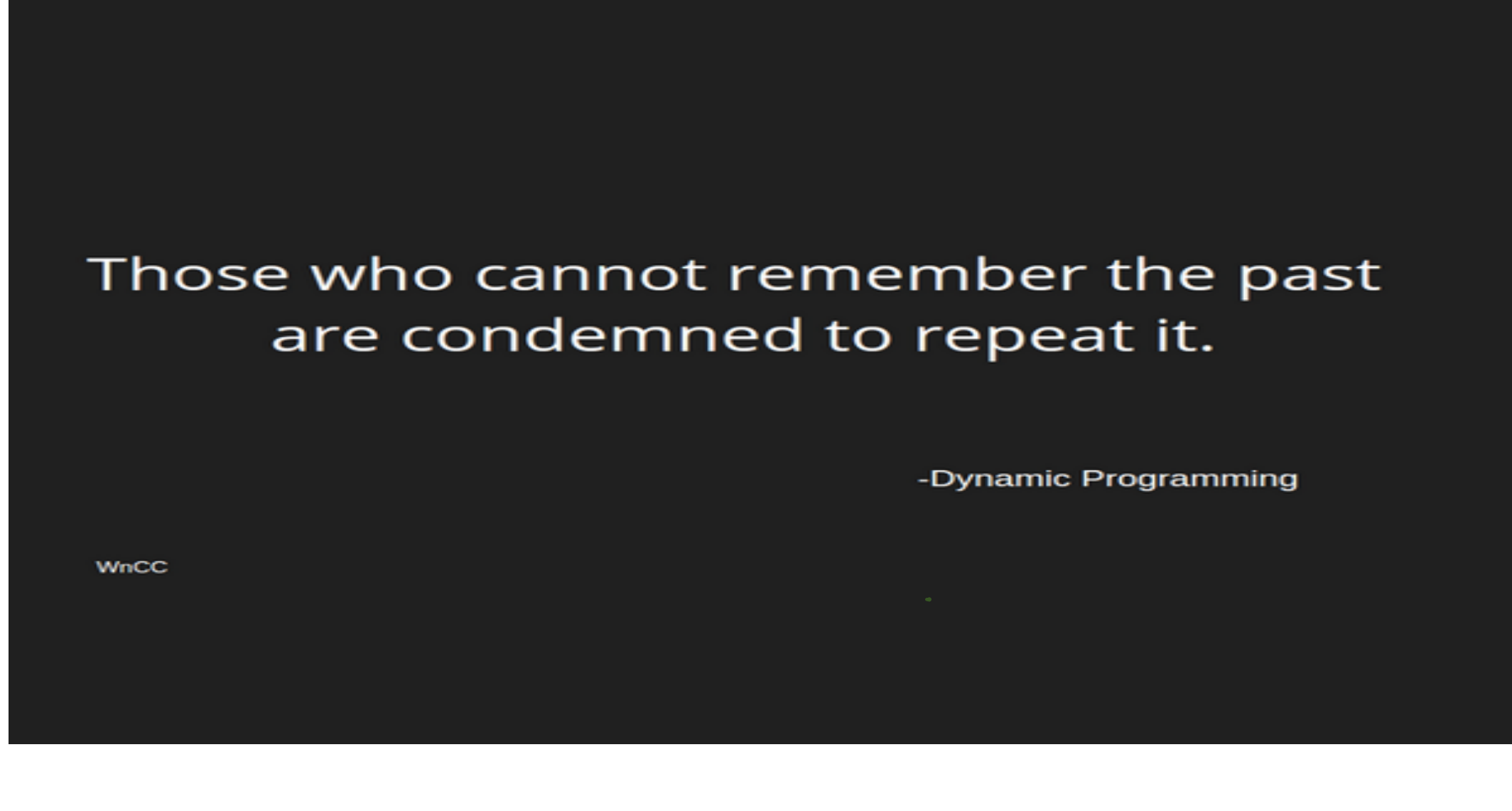
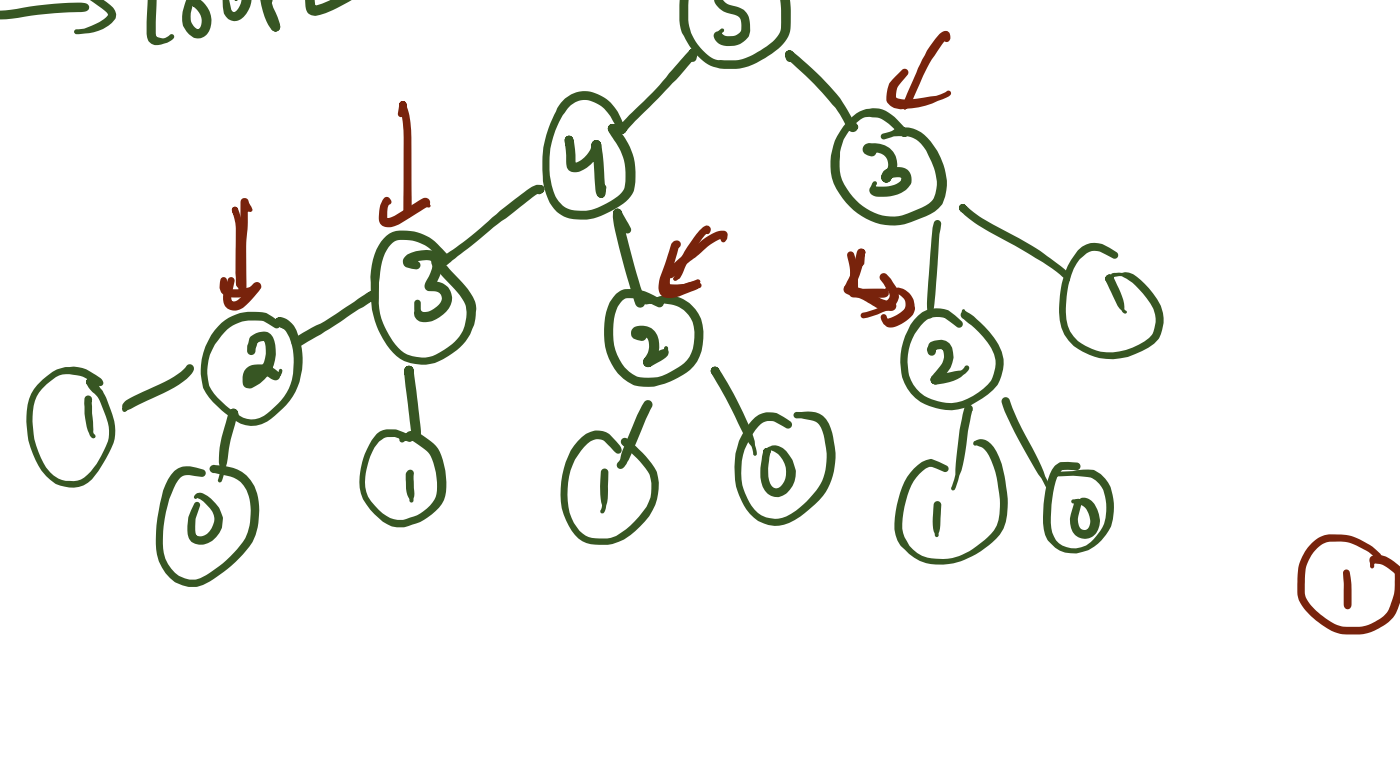




D.P X
↳ problem

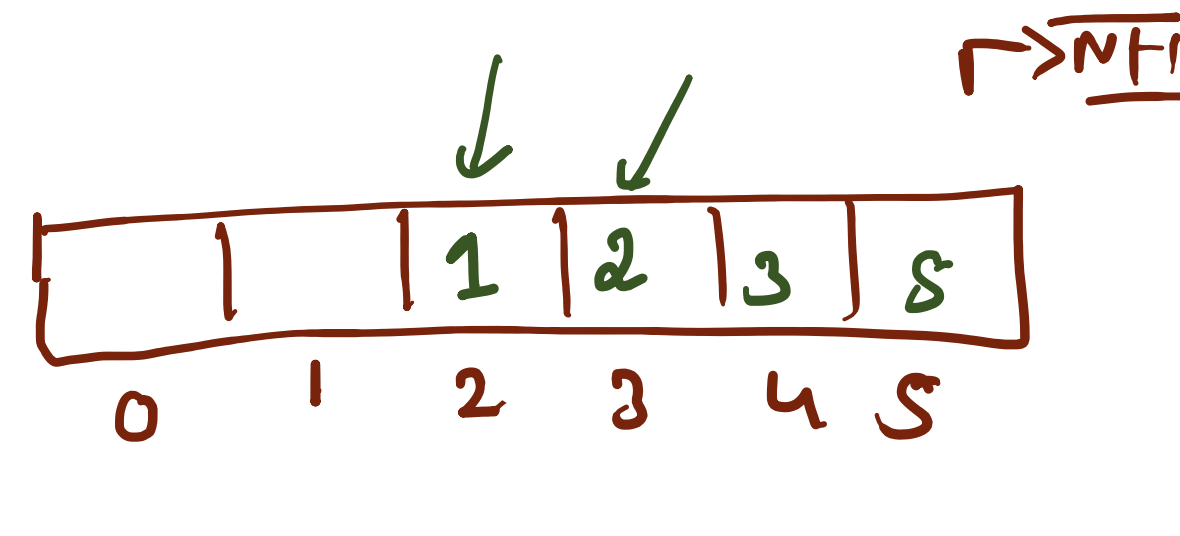
① D.P ✓
② Solve
Recursive
Iterative method

- ① Top-down → Recursive → memoization
- ② Bottom-up → Iterative → loop

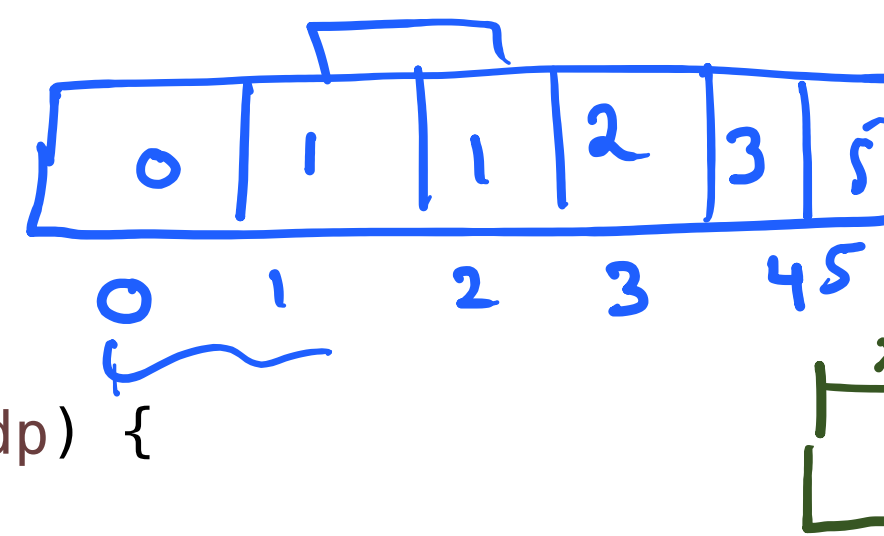


- ① Optimal substructure
- ② Problem → overlapping

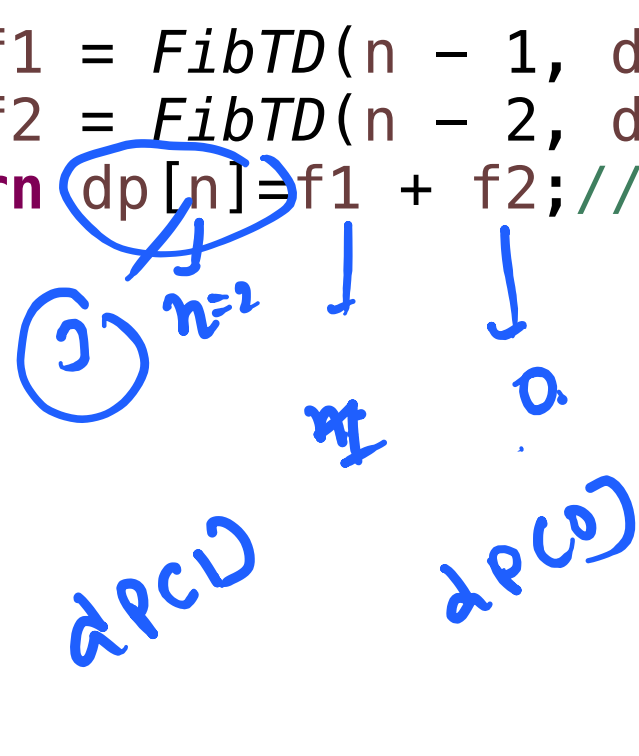
```
public static int Fib(int n) {  
    if (n == 0 || n == 1) {  
        return n;  
    }  
    int f1 = Fib(n - 1);  
    int f2 = Fib(n - 2);  
    return f1 + f2;  
}
```



$dp[i] = dp[i-1] + dp[i-2]$
[i=2]

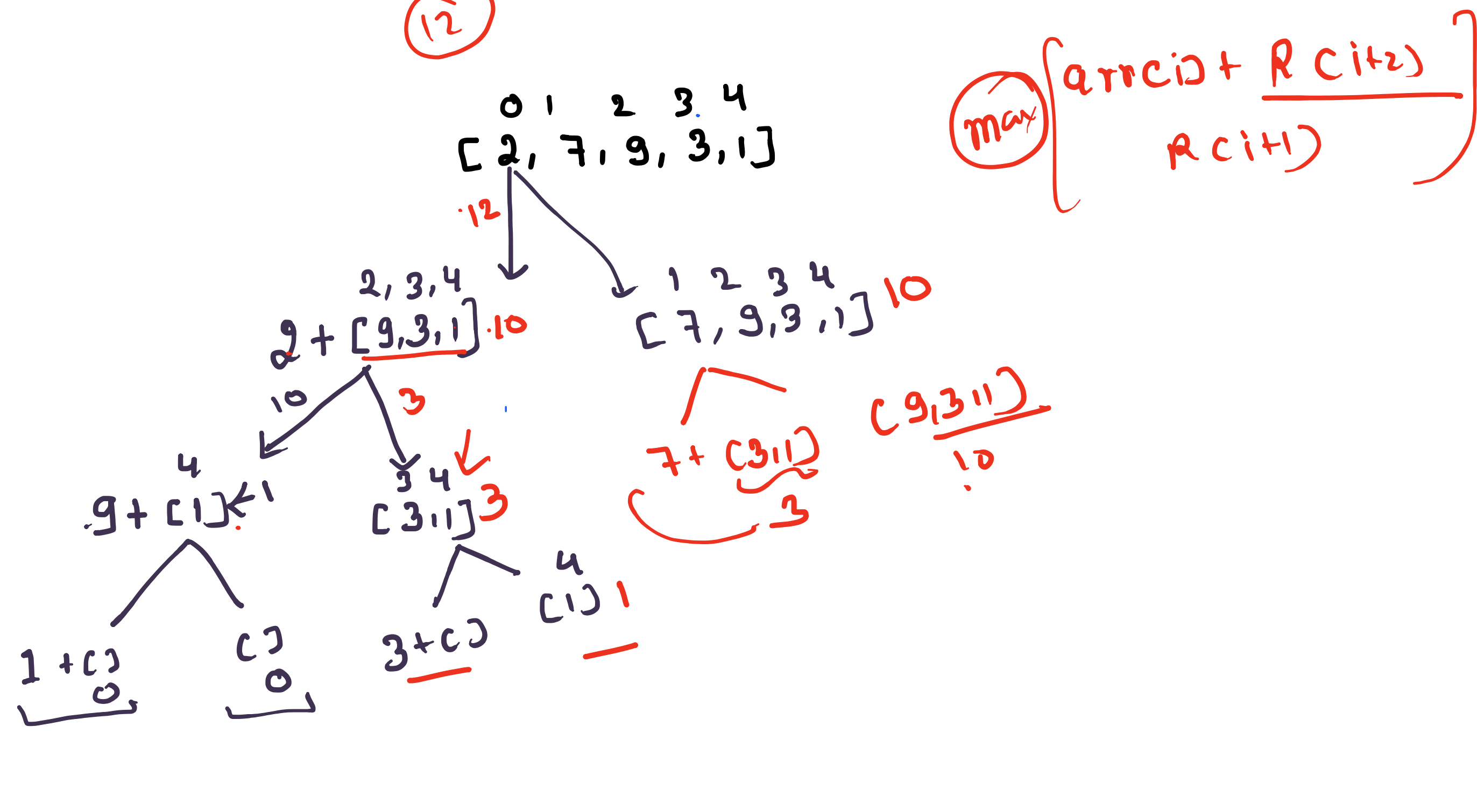


```
public static int FibTD(int n, int[] dp) {  
    if (n == 0 || n == 1) {  
        return n;  
    }  
    if (dp[n] != 0) {  
        return dp[n];  
    }  
    int f1 = FibTD(n - 1, dp);  
    int f2 = FibTD(n - 2, dp);  
    return dp[n] = f1 + f2; // yaad kra hai  
}
```

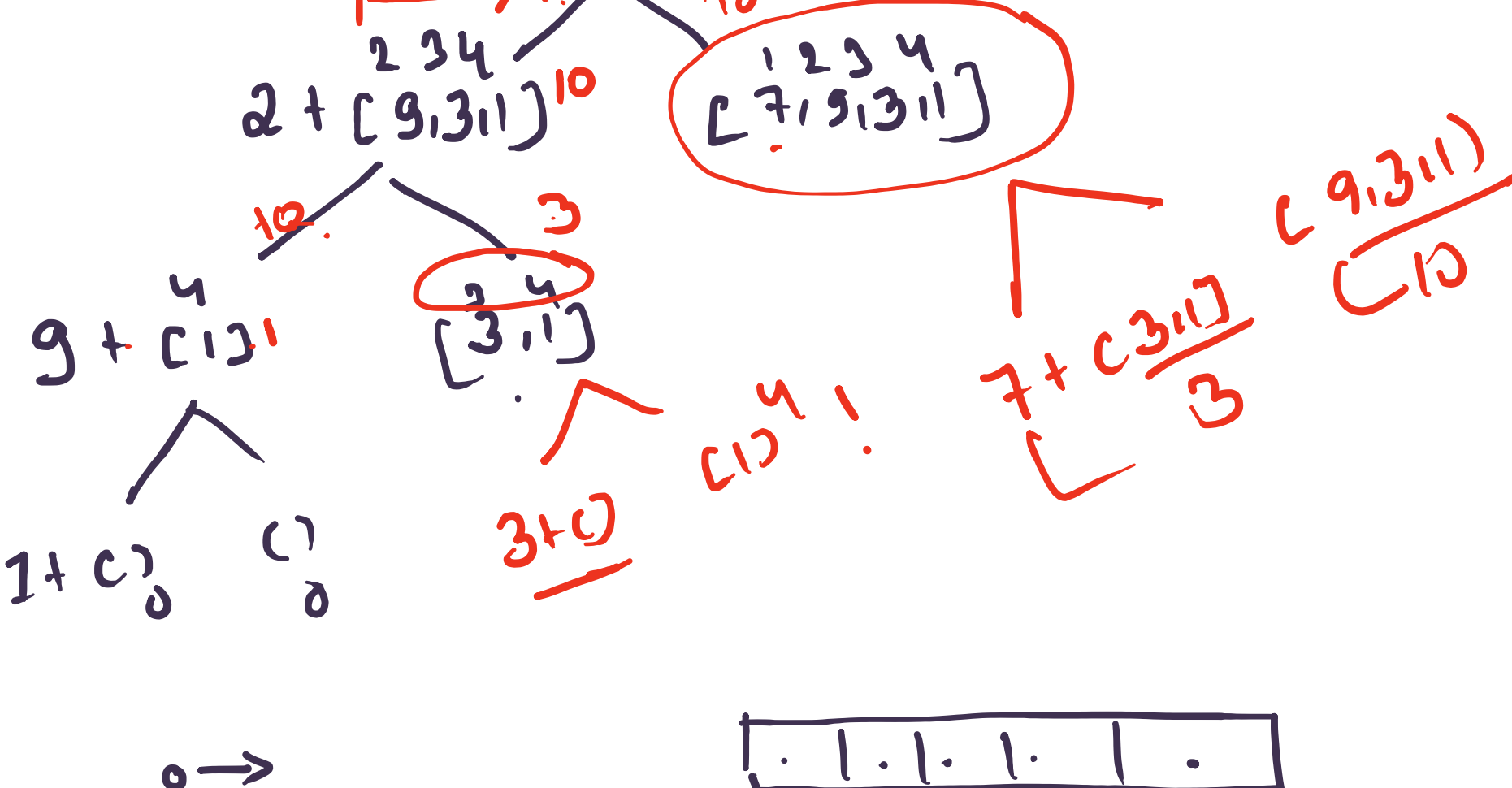
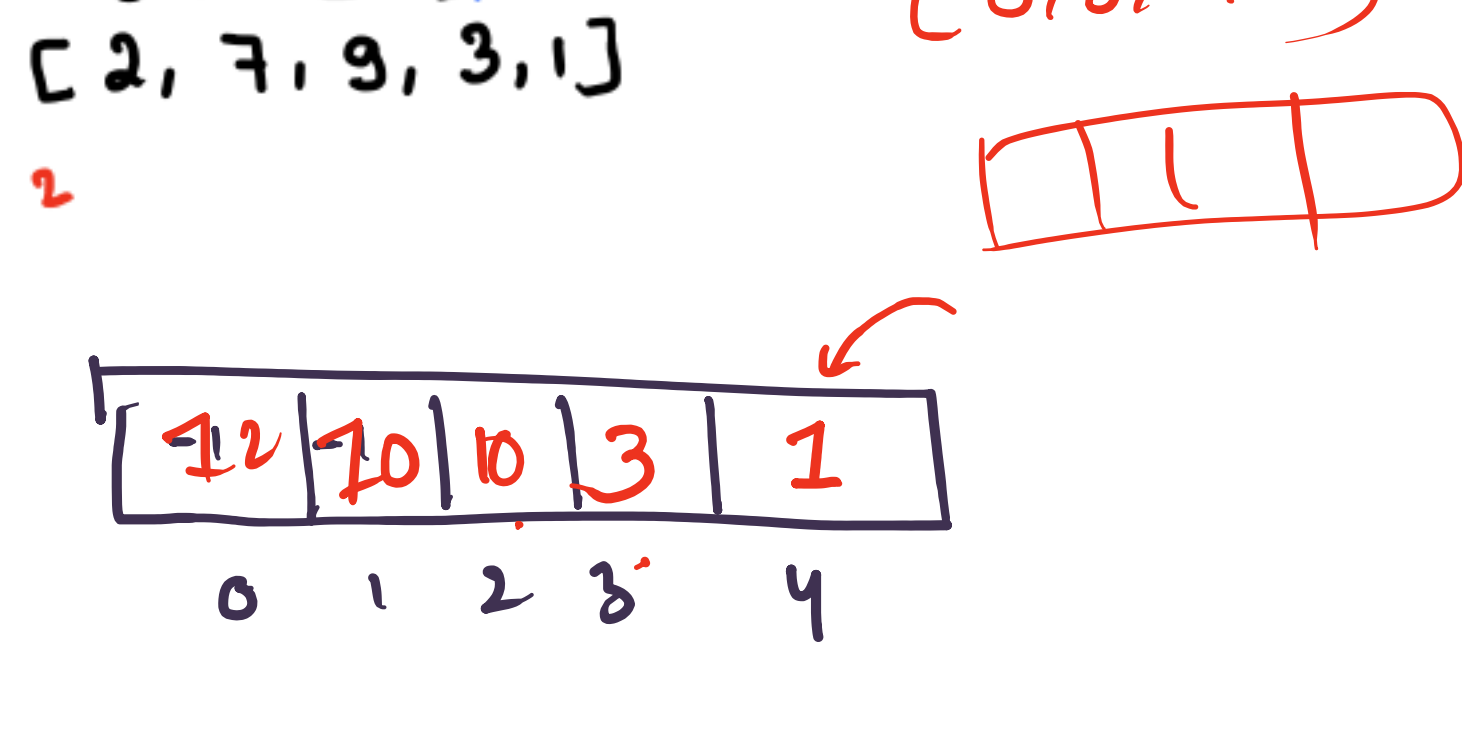


$dp[2] = dp[1] + dp[0]$
 $dp[3] = dp[2] + dp[1]$
 $dp[4] = dp[3] + dp[2]$

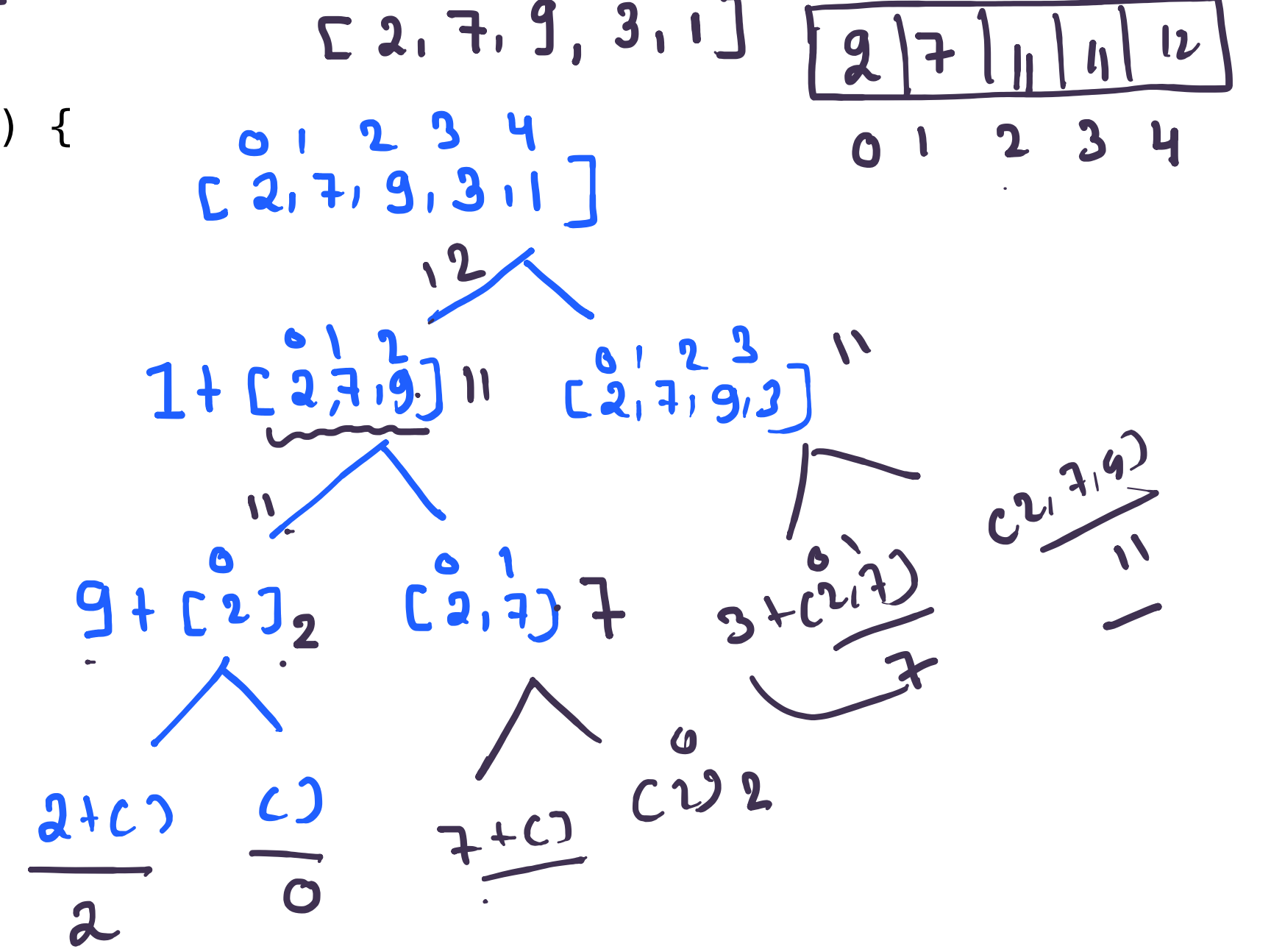
[2, 7, 9, 3, 1]



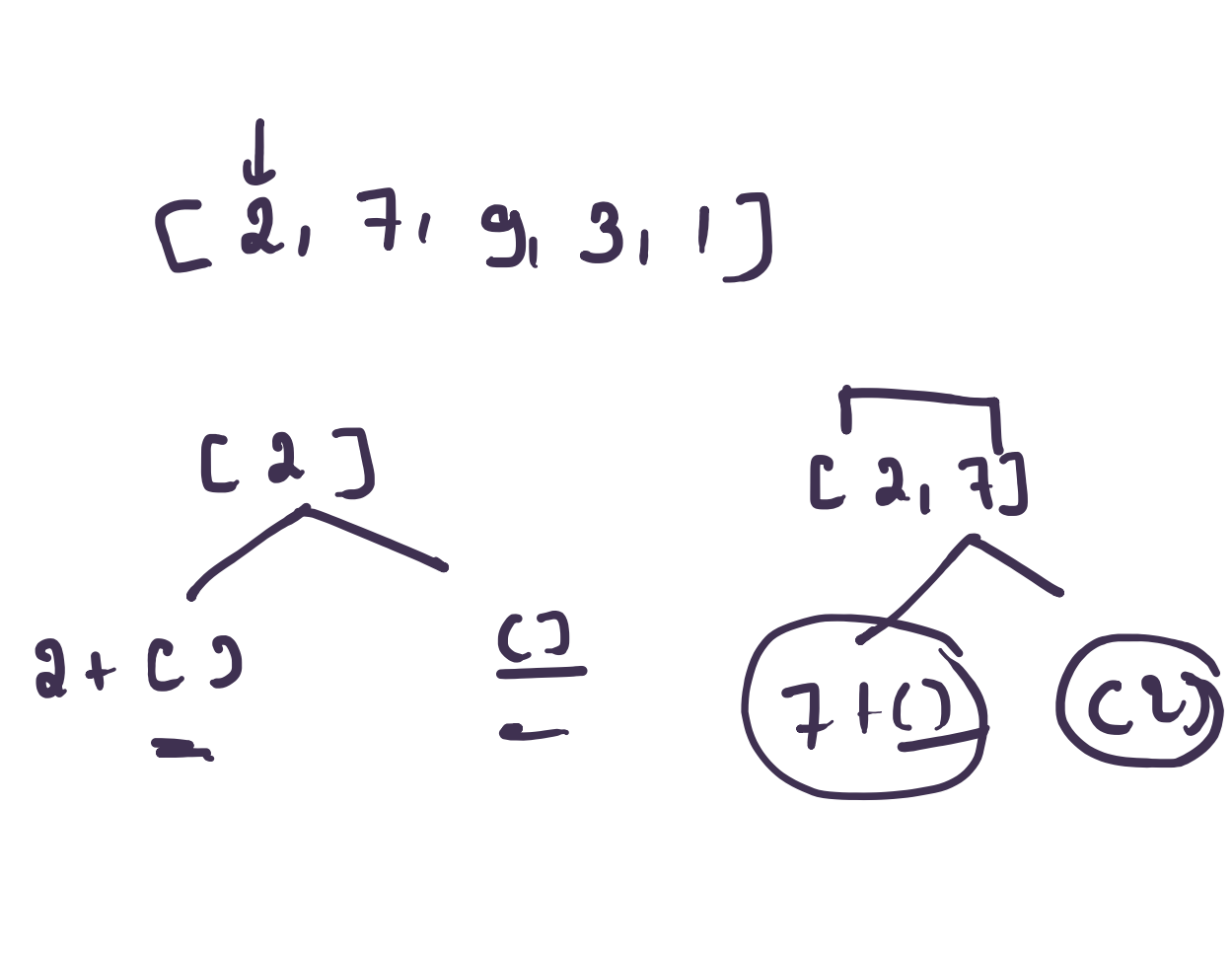
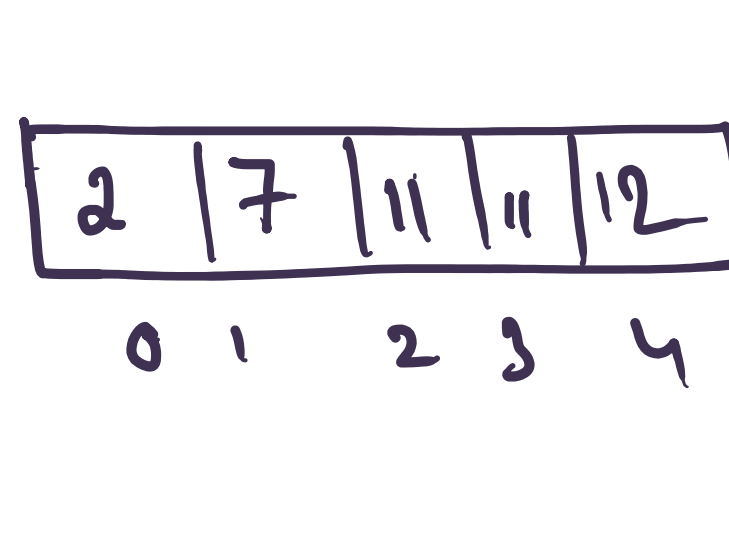
```
public static int Robber(int[] arr, int i) {  
    if (i >= arr.length) {  
        return 0;  
    }  
    int rob = arr[i] + Robber(arr, i + 2);  
    int dont_rob = Robber(arr, i + 1);  
    return Math.max(rob, dont_rob);  
}
```



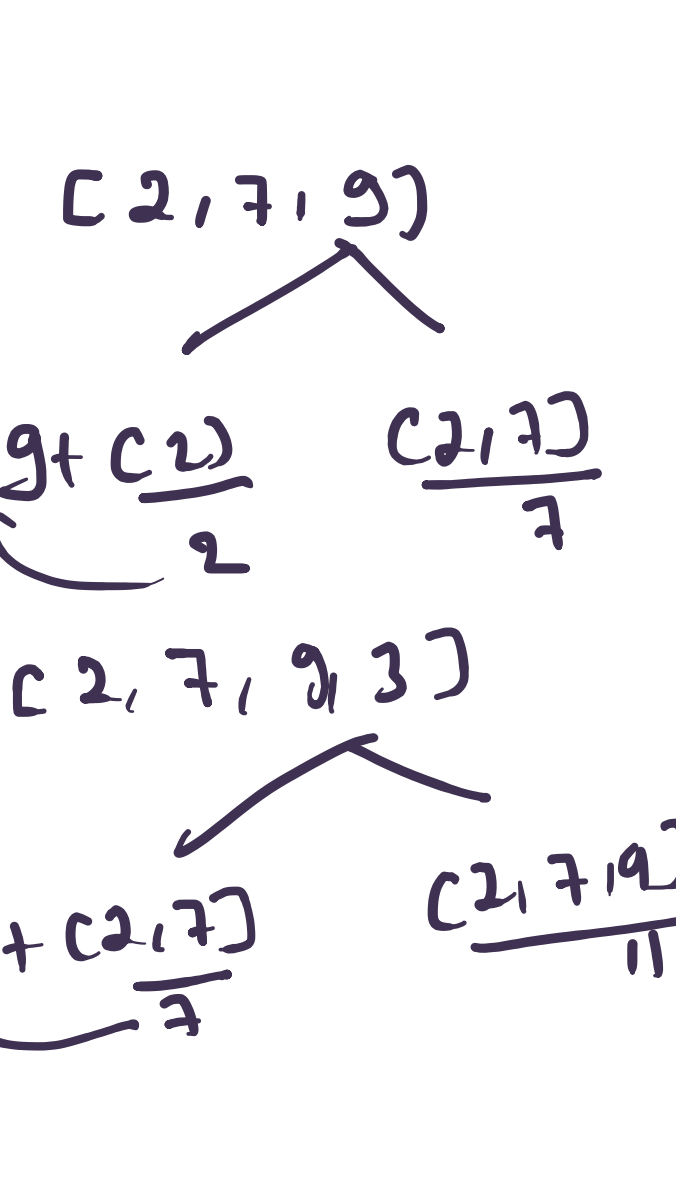
```
public static int Robber(int[] arr, int i, int[] dp) {  
    if (i >= arr.length) {  
        return 0;  
    }  
    if (dp[i] != -1) {  
        return dp[i];  
    }  
    int rob = arr[i] + Robber(arr, i + 2, dp);  
    int dont_rob = Robber(arr, i + 1, dp);  
    return dp[i] = Math.max(rob, dont_rob);  
}
```



$dp[i] = \max(dp[i-1], dp[i-2] + arr[i])$



$dp[0] = arr[0]$
 $dp[i] = \max(dp[i-1], dp[i-2] + arr[i])$



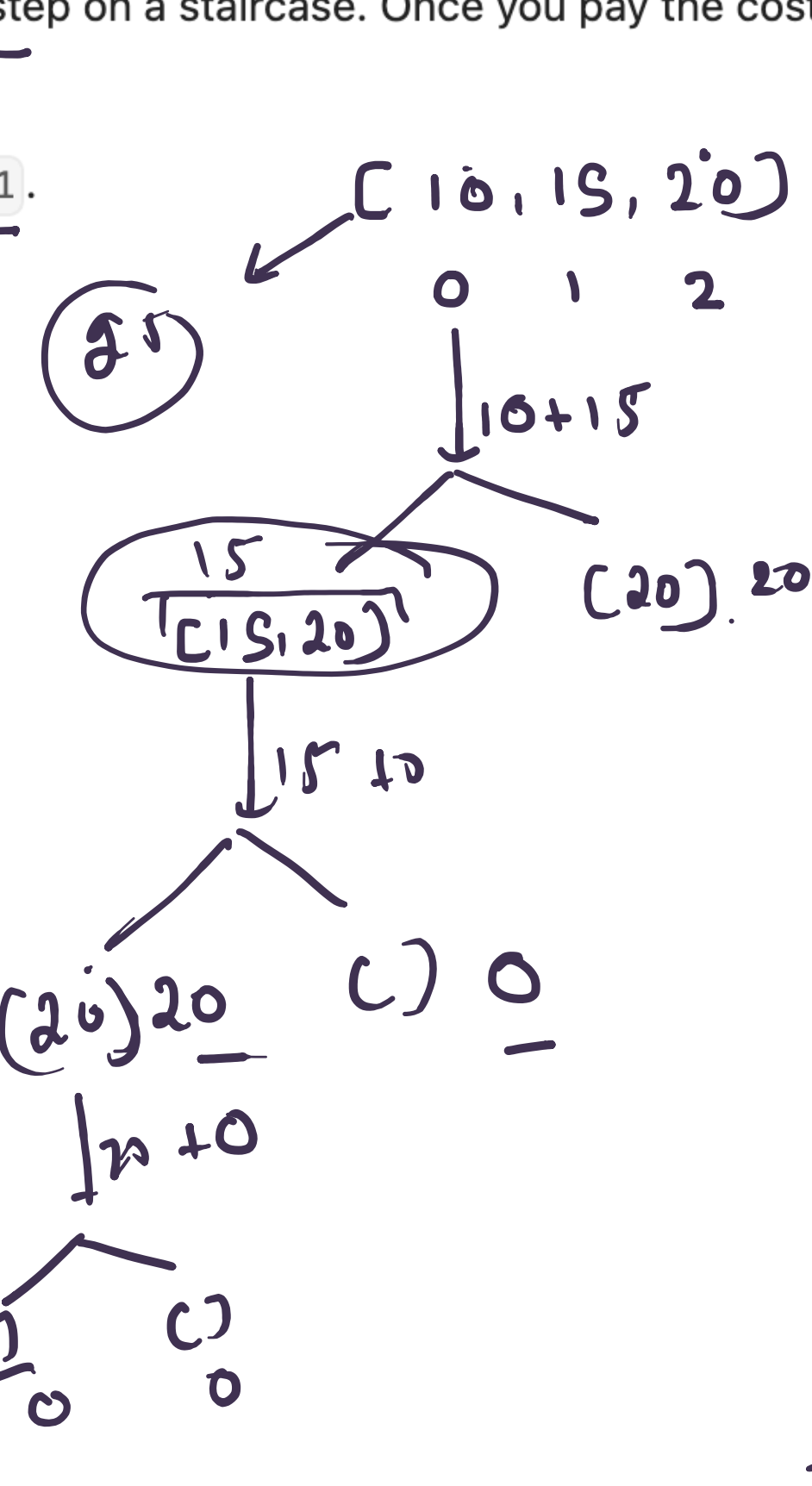
You are given an integer array cost where cost[i] is the cost of ith step on a staircase. Once you pay the cost, you can either climb one or two steps.

You can either start from the step with index 0, or the step with index 1.

Return the minimum cost to reach the top of the floor.

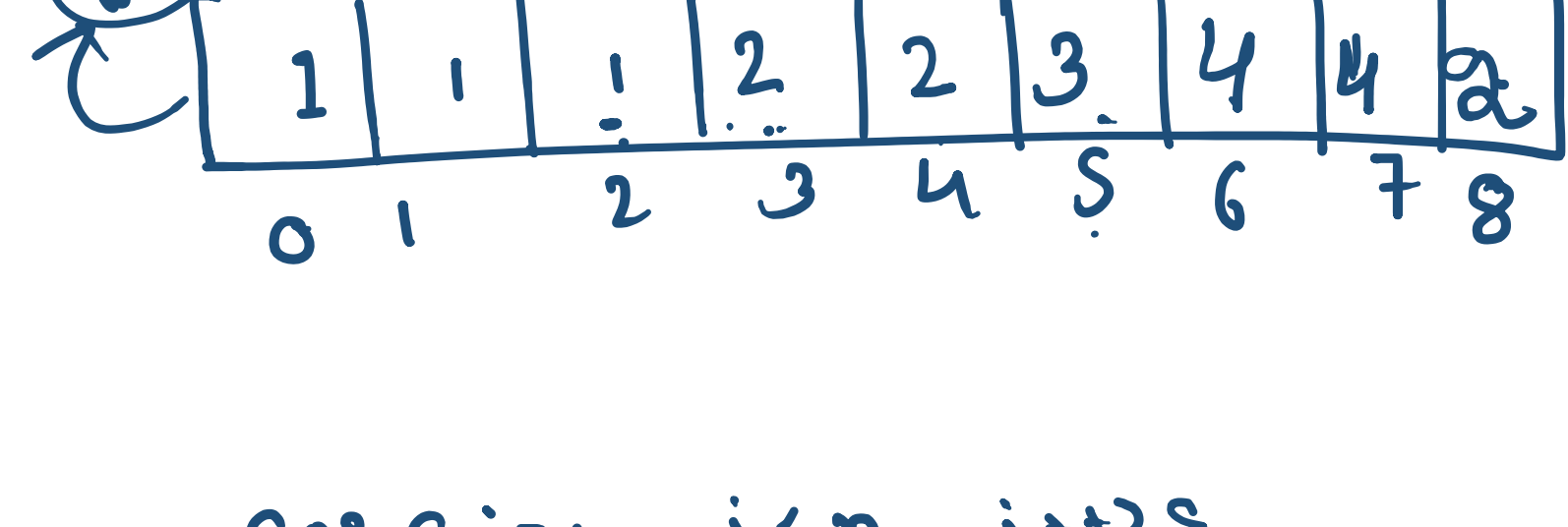
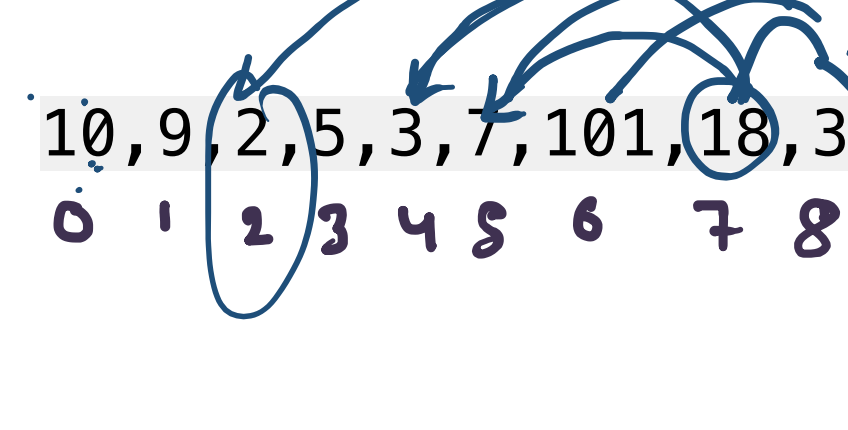
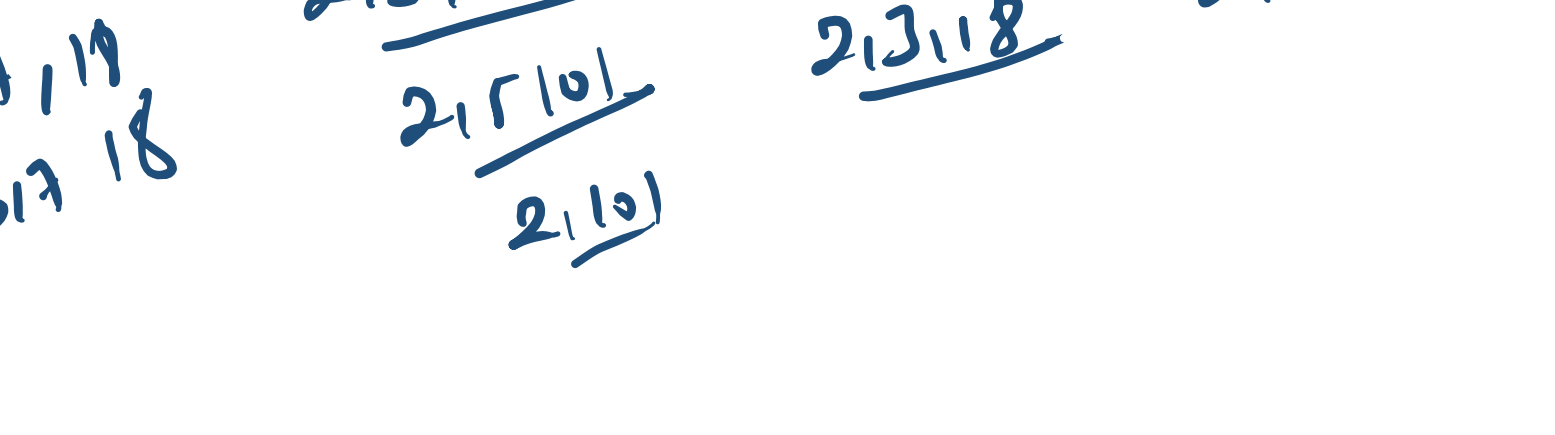
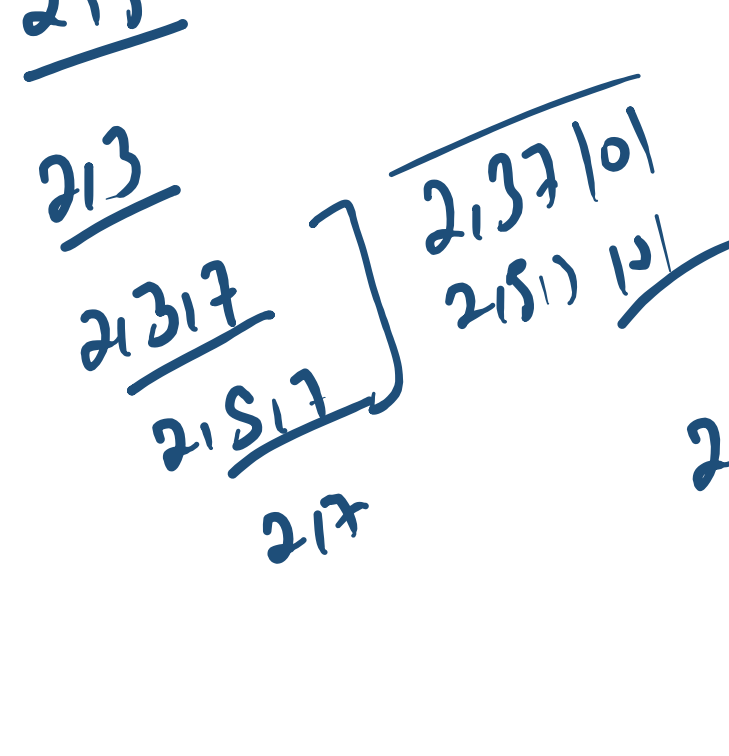
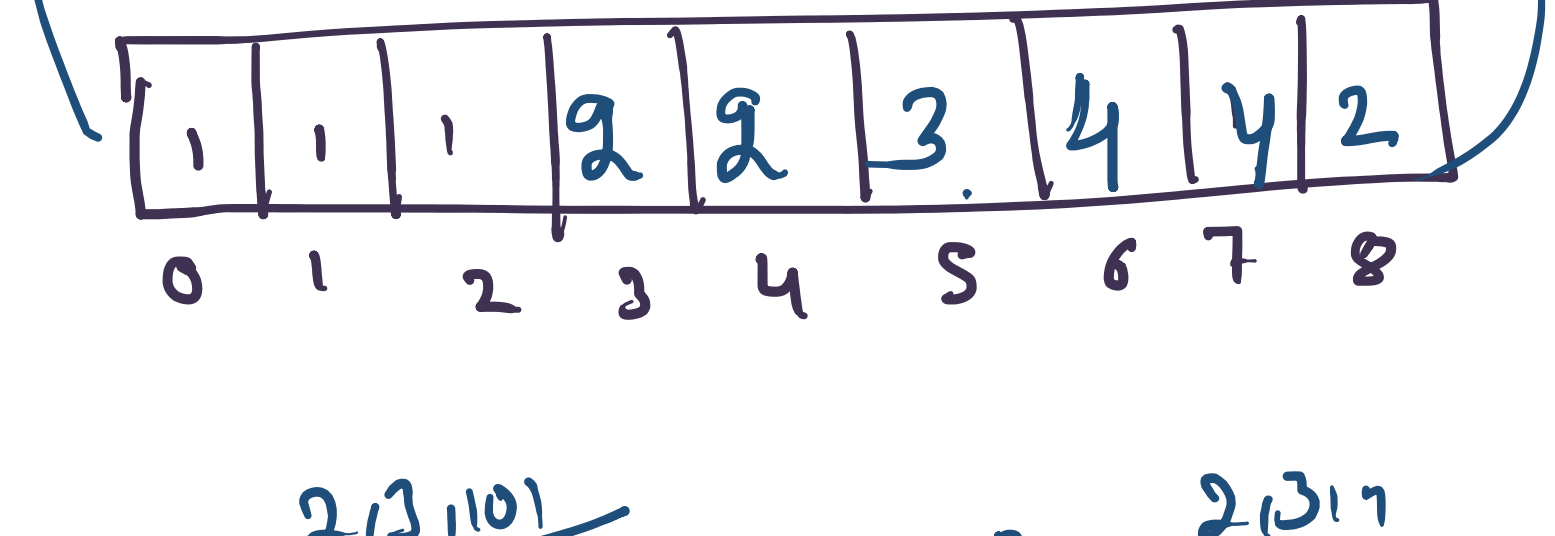
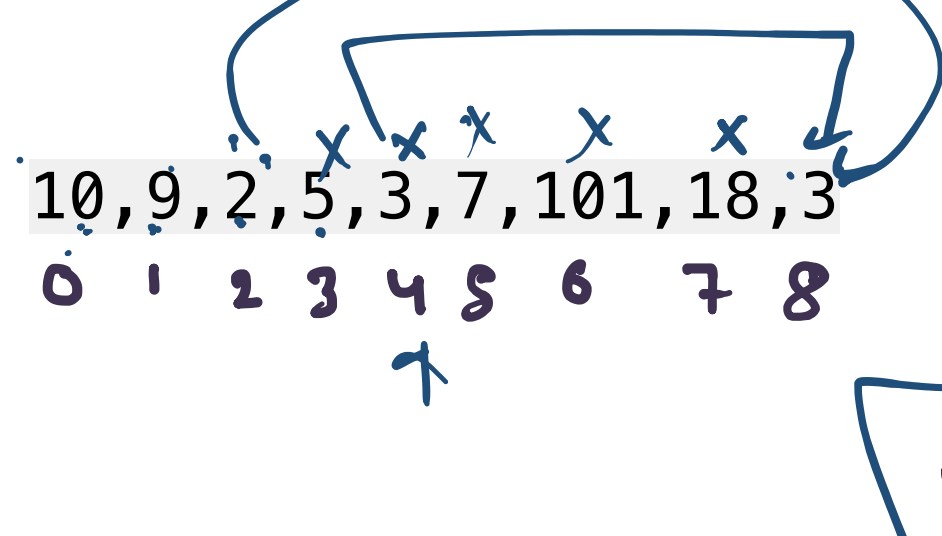
Example 1:

Input: cost = [10, 15, 20]
Output: 15
Explanation: You will start at index 1.
- Pay 15 and climb two steps to reach the top.
The total cost is 15.



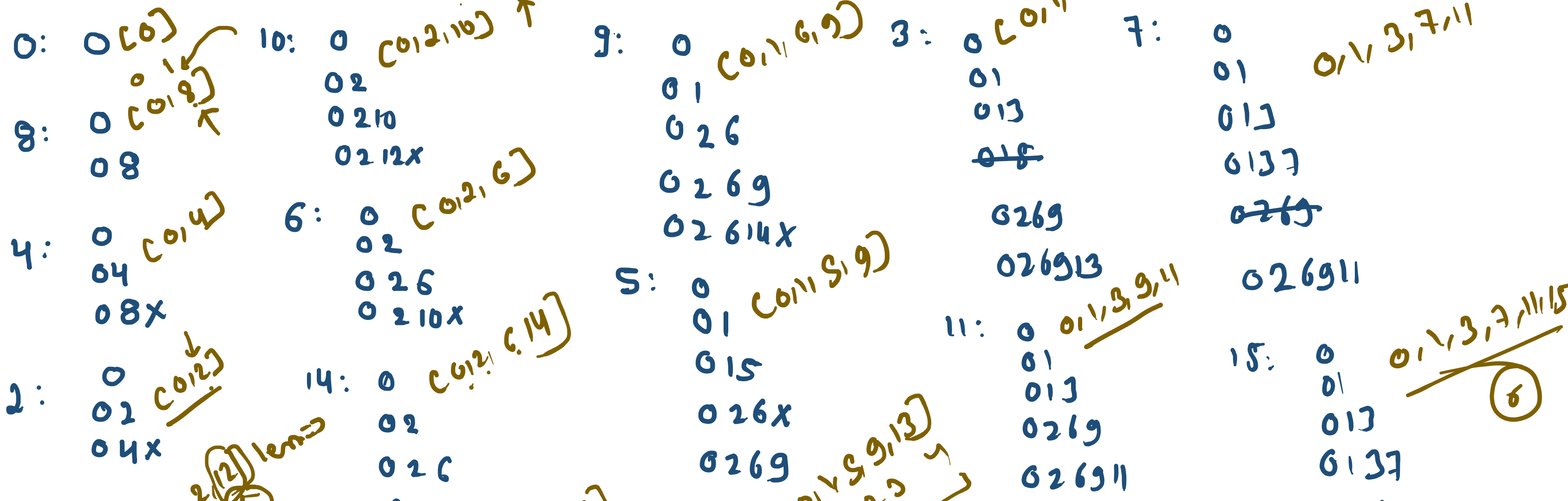
[15]

5 4 3 2 1



$x = dp[2] \rightarrow 1$
 $2, 5 \rightarrow 2$
 $2, 4 \rightarrow 3$
 $2, 4 \rightarrow 3$
 $2, 11 \rightarrow 2$

for i=1 i < n i++
for j=1-1 j >= 0 j--
if (arr[i] > arr[j]) {
 x = dp[j]
 dp[i] = max(dp[i], x+1)
}



```
if (arr[i] > dp[len-1]) {  
    dp[len] = arr[i];  
    len++;  
}
```